TAXDOME · TECHNICAL CHALLENGE

Firm Payments: Build a bulk payment service

Write a web service that receives bulk payment requests from a single firm

© TaxDome 2026. All rights reserved.

Firm Payments: Build a bulk payment service.

Goal

Imagine you are a backend engineer at a practice management platform for accounting firms, like TaxDome. Thousands of firms work on the platform, from solo preparers to practices with hundreds of accountants. Each firm has a balance with us, and the money its clients pay on invoices lands there.

Firms also work with each other. A large practice sends overflow returns to solo preparers during tax season, a bookkeeping firm hands its clients' tax returns to a preparer, and a firm that brings in a client gets a referral fee. Firms settle these debts through their balances on the platform, so the same firm pays some firms and gets paid by others. Paying everyone one by one would be slow and painful, so we let a firm pay everyone it owes in bulk.

Your mission

Write a web service that receives bulk payment requests from a single firm and verifies that the request is valid: **the paying firm must have enough funds on its balance for all the payments in the request.** If it doesn't, the entire request is denied.

422 DENIED

If the request is denied, return a 422 HTTP response.

201 CREATED

Otherwise, add the payments to the database, debit the paying firm's balance, credit each receiving firm's balance, and return a 201 HTTP response.

Take care to consider

- Assume the server runs as **multiple load-balanced instances**.
- Pretend you are using PostgreSQL, MySQL or a similar relational database.

Instructions

LANGUAGE	Use one of: **Ruby** **Python** **Go**
AI & WEB	You can use any AI tools — in this case please include **unedited logs** of your conversations with the LLM. Don't clean them up or summarize; the raw back-and-forth is exactly what we'd like to see.
LIBRARIES	Prefer commonly used libraries whenever possible.

© TaxDome 2026 · TaxDome Interview Flow

Request format

A bulk payment request is a JSON document. Here is an example, with each attribute explained in a comment (real requests carry no comments):

```
{
  // Identifies the firm that pays.
  "payer_firm_uuid": "3f1c9a2e-7b4d-4c1e-9a55-2d8e6f0b7c41",
  // Each entry is one payment to one firm.
  "payments": [
    {
      // Amount of the payment in US dollars, as a string, with at most
      // 2 optional decimal places. Always positive.
      "amount": "1200.75",
      // Identifies the firm that receives the payment.
      "payee_firm_uuid": "8b2e4c71-0d3a-4f6e-b1c9-5a7d2e9f4c10",
      // Description of the payment.
      "description": "Bookkeeping cleanup, 3 clients"
    },
    {
      "amount": "300",
      "payee_firm_uuid": "e5f18b3c-2a9d-4c07-8e6b-1d4a7f9c3b25",
      "description": "Referral fee, 2 clients"
    }
  ]
}
```

The platform's database

The database has two tables.

`firms`

Column	Type	Description
`id`	INTEGER	Unique internal identifier of the firm.
`name`	TEXT	Name of the firm.
`balance_cents`	INTEGER	Current balance of the firm, in US cents.
`uuid`	TEXT	Public identifier of the firm, used in requests.

`payments`

Column	Type	Description
`id`	INTEGER	Unique identifier of the payment.
`payer_firm_id`	INTEGER	id of the firms row that pays.
`payee_firm_id`	INTEGER	id of the firms row that receives the payment.
`amount_cents`	INTEGER	Amount of the payment, in US cents.
`description`	TEXT	Description of the payment.

Sample request

Seed the database with three firms:

```
INSERT INTO firms (id, name, balance_cents, uuid)
VALUES (1, 'Pinecrest CPA Group', 5000000, '3f1c9a2e-7b4d-4c1e-9a55-2d8e6f0b7c41'),
       (2, 'Lopez Bookkeeping', 50000, '8b2e4c71-0d3a-4f6e-b1c9-5a7d2e9f4c10'),
       (3, 'Nair Tax Services', 200000, 'e5f18b3c-2a9d-4c07-8e6b-1d4a7f9c3b25');
```

Pinecrest pays Nair and Lopez. It can cover the **$13,251.25** total, so the service returns **201**. Afterwards Pinecrest has $36,748.75, Lopez has $1,700.75 and Nair has $14,050.50.

Firm	Balance before	Balance after
Pinecrest CPA Group	$50,000.00	**$36,748.75**
Lopez Bookkeeping	$500.00	**$1,700.75**
Nair Tax Services	$2,000.00	**$14,050.50**

```
{
  "payer_firm_uuid": "3f1c9a2e-7b4d-4c1e-9a55-2d8e6f0b7c41",
  "payments": [
    {
      "amount": "6250",
      "payee_firm_uuid": "e5f18b3c-2a9d-4c07-8e6b-1d4a7f9c3b25",
      "description": "Overflow returns, August 2026"
    },
    {
      "amount": "5800.5",
      "payee_firm_uuid": "e5f18b3c-2a9d-4c07-8e6b-1d4a7f9c3b25",
      "description": "Amended returns, August 2026"
    },
    {
      "amount": "1200.75",
      "payee_firm_uuid": "8b2e4c71-0d3a-4f6e-b1c9-5a7d2e9f4c10",
      "description": "Bookkeeping cleanup, 3 clients"
    }
  ]
}
```

What to deliver

3 ITEMS

1. **Source code**
   A repository (GitHub, GitLab or similar, either public or shared with us) containing your service. We should be able to clone it, install it, run it, and send the sample requests to it. How you choose to verify the service is part of the design. What level of testing it needs is your call.

2. **How you worked**
   Include your commit history. If you used AI tools, show us how you used them: exported chats, transcripts or prompt-and-response pairs from your sessions. Name the tools (Copilot, Cursor, Claude Code, aider or others) and say where each one fit into your workflow. Any tool is fine.

3. **README**
   Write it as if you were opening a pull request for your teammates to review. Cover: your solution and the approach you took, the issues you ran into, the assumptions you made, possible improvements, and how to run and verify it.

Two quick questions

At the end of the README, please also answer:

How much time did you spend on this task?

- ○ Less than 3h
- ○ 3h to 6h
- ○ More than 6h (how long?)

How proud are you of your work?

- ○ Very proud
- ○ Fairly proud
- ○ Good enough
- ○ Somewhat disappointed

When you're done, send us the link to the repository.

No "perfect" solution expected

We're interested in your thought process and your approach to the problem, and we'll consider every submission, even an incomplete one. If you'd like to take extra time to improve or "finish" your solution, let us know beforehand and we'll review the final version.

© TaxDome 2026 · TaxDome Interview Flow